



SGCI

Frontend — Documentação Técnica Aprofundada

SPA em Angular 20 · Standalone + Signals · Análise do código-fonte

Angular 20.3

Standalone Components

Signals

RxJS 7.8

Atualizado em 12/07/2026

Sumário

1. Panorama e princípios de design
2. Bootstrap e configuração global (`main.ts` , `app.config.ts`)
3. Estrutura de pastas
4. Camada `core` — configuração e modelos
5. Autenticação no cliente (`AuthService` + signals)
6. Interceptor de credenciais
7. Guards de rota (auth e admin)
8. Diretiva estrutural `*appRole`
9. Roteamento e lazy loading
10. Layout principal (sidebar agrupada, header, busca, notificações)
11. Features — visão geral e padrão comum
12. Feature Crises em profundidade (form + máquina de estados)
13. Dashboard por perfil, Relatórios e Auditoria
14. Primitivos de UX (Toast, Modal, Skeleton)
15. Primeiro acesso (modal bloqueante) e recuperação de senha
16. Meu Perfil
17. Dashboard por perfil e Centro de Monitoramento (admin)
18. Gráficos em SVG/CSS sem biblioteca
19. Acesso Negado e endurecimento do perfil COMUM
20. Fluxos de ponta a ponta

1. Panorama e princípios de design

O frontend do SGCI é uma **Single Page Application em Angular 20.3**, 100% standalone (sem `NgModule`), com estado reativo baseado em **signals** e um design system próprio em SCSS. Ele apenas renderiza as telas e consome a API REST do backend Quarkus.

Seis características que definem o front

#	Característica	Consequência prática
1	100% standalone — todo componente declara seus próprios <code>imports</code> .	Não há <code>AppModule</code> ; o bootstrap é direto no componente raiz.
2	Signals para estado (<code>signal</code> , <code>computed</code> , <code>effect</code>).	Reatividade fina sem <code>BehaviorSubject</code> ; a UI reage a login/logout automaticamente.
3	<code>inject()</code> em vez de injeção por construtor.	Código mais enxuto nos componentes/serviços.
4	Lazy loading em toda rota (<code>loadComponent</code>).	Cada tela vira um chunk carregado sob demanda.
5	Sessão por cookie (HttpOnly), não token no JS.	O cliente só espelha o usuário; a sessão real vive no servidor.
6	Sem biblioteca de UI (nada de Material/Bootstrap).	Design system manual em <code>styles.scss</code> , ícones SVG inline e gráficos SVG/CSS próprios .

Stack

Tecnologia	Versão	Papel
Angular	20.3	Framework SPA (core, common, forms, router)
TypeScript	5.9	Linguagem
RxJS	7.8	Observables para HTTP, <code>forkJoin</code> e eventos de navegação
@angular/build	20.3	Application builder (novo)
SCSS	—	Design system próprio
Karma + Jasmine	—	Testes

2. Bootstrap e configuração global

A aplicação sobe sem módulos: `main.ts` monta o componente raiz `App` com um objeto de configuração. O `App` contém um `<router-outlet>` e monta uma única vez o `<app-toast-container>` (pilha global de notificações — ver seção 14).

```
// main.ts
bootstrapApplication(App, appConfig).catch((err) => console.error(err));

// app.ts - componente raiz
@Component({
  selector: 'app-root',
  imports: [RouterOutlet, ToastContainerComponent], // toasts globais
  templateUrl: './app.html',
})
export class App {}
```

Todos os provedores globais ficam em `app.config.ts` :

```
// app.config.ts
export const appConfig: ApplicationConfig = {
  providers: [
    provideBrowserGlobalErrorListeners(),
    provideZoneChangeDetection({ eventCoalescing: true }),
    provideRouter(routes),
    provideHttpClient(withInterceptors([credentialsInterceptor])), // cookie em toda req.
  ],
};
```

Pontos importantes:

- `provideRouter(routes)` — registra as rotas lazy (seção 9).
- `provideHttpClient(withInterceptors([ ... ]))` — o `HttpClient` passa por um interceptor funcional que injeta as credenciais (o cookie de sessão) em toda requisição.
- `provideZoneChangeDetection({ eventCoalescing:true })` — detecção de mudanças baseada em Zone, agrupando eventos para reduzir ciclos.

A base da API está centralizada em `core/config/api.config.ts` : `export const API_BASE_URL = 'http://localhost:8080'`; — todos os serviços derivam suas URLs daí.

3. Estrutura de pastas

```
app/
├─ app.ts / app.config.ts / app.routes.ts (raiz; app.ts monta o ToastContainer)
├─ core/                                infraestrutura transversal
│   ├─ config/      api.config.ts      (API_BASE_URL)
│   ├─ guards/      auth.guard · admin.guard
│   ├─ interceptors/ credentials.interceptor
│   ├─ directives/  role.directive      (*appRole)
│   ├─ models/      auth.model · user.model · profile.model
│   ├─ utils/       http-error.ts · password-policy.ts
│   ├─ components/  primitivos e gráficos reutilizáveis
│   │   ├─ toast/   toast-container.component
│   │   └─ modal/   modal.component
```

```

|   |   └ charts/    pie-chart · line-chart · bar-chart · chart.types
|   └ services/     auth.service · toast.service
└ layouts/
  └ main-layout/    casca das telas autenticadas (sidebar + header + modais)
└ features/         uma pasta por domínio
  └ auth/
    └ pages/        login · forgot-password · reset-password
    └ components/   change-password-modal
  └ dashboard/      pages/ dashboard · admin-dashboard
  └ perfil/         pages/ perfil (Meu Perfil)
  └ acesso-negado/  acesso-negado.component (destino do adminGuard)
  └ crises/        list · form · detail
  └ relatorios/    list · form
  └ auditoria/     list (admin)
  └ usuarios/      list · form (admin)
  └ departamentos · tipos-crise · instituicoes · campi · cenarios
    (cada feature: models/ · pages/ · services/)

```

Cada feature isola `models` (interfaces/tipos), `pages` (componentes de tela) e `services` (acesso HTTP), mantendo alto o acoplamento interno e baixo entre features. O que é compartilhado por várias features — primitivos de UX, gráficos, utilitários — foi promovido a `core/components` e `core/utils`.

4. Camada `core` — configuração e modelos

O pacote `core` reúne tudo que é transversal: config da API, guards, interceptor, diretiva de papel, os serviços de autenticação/toast, utilitários e os modelos base.

MODELOS DE AUTENTICAÇÃO E USUÁRIO

```
// auth.model.ts
export interface LoginRequest { email: string; password: string; }
export interface LoginResponse { authenticated: boolean; message: string; mustChangePassword: boolean; }
export interface ChangePasswordRequest { currentPassword: string; newPassword: string; }
export interface ForgotPasswordRequest { email: string; }
export interface ResetPasswordRequest { token: string; newPassword: string; }

// user.model.ts
export type Perfil = 'ADMIN' | 'COMUM';
export interface User { id: number; name: string; email: string;
  perfil: Perfil; mustChangePassword: boolean; } // flag de primeiro acesso
```

O campo `LoginResponse.mustChangePassword` e `User.mustChangePassword` alimentam o fluxo de **primeiro acesso** (seção 15): quando verdadeiro, o layout exibe um modal bloqueante de troca de senha.

MODELO DE PERFIL (MEU PERFIL)

```
// profile.model.ts - dados cadastrais somente leitura
export interface Profile {
  id: number; name: string; email: string; perfil: Perfil;
  cargo: string | null;
  departamentoId: number | null; departamentoNome: string | null;
  instituicaoId: number | null; instituicaoNome: string | null;
}
```

O backend resolve os *nomes* de departamento e instituição, então a tela de perfil apenas exibe strings já prontas, sem novas chamadas.

POLÍTICA DE SENHA (`CORE/UTILS/PASSWORD-POLICY.TS`)

Regra compartilhada entre a troca (primeiro acesso e "Meu Perfil") e a redefinição de senha:

```
export const PASSWORD_MIN_LENGTH = 8;
export function validatePasswordPolicy(password: string): string | null {
  if (password.length < PASSWORD_MIN_LENGTH) return `A senha deve ter ao menos ${PASSWORD_MIN_LENGTH} caracteres.`;
  if (!/[A-Za-z]/.test(password)) return 'A senha deve conter ao menos uma letra.';
  if (!/[0-9]/.test(password)) return 'A senha deve conter ao menos um número.';
  return null; // aprovada
}
```

Mín. 8 caracteres + ao menos uma letra + um número. A validação retorna uma mensagem amigável (ou `null` se ok) e está alinhada à política do backend (Bean Validation nos DTOs de senha).

Os tipos do front espelham exatamente os DTOs do backend — por exemplo, `Perfil` reflete o enum `PerfilUsuario`, e cada feature tem seus próprios tipos (ex.: `StatusCrise`, `NivelCriticidade`, `TipoAcao`) idênticos aos enums Java.

UTILITÁRIO DE ERRO HTTP (`CORE/UTILS/HTTP-ERROR.TS`)

`extractApiError(err, fallback)` traduz uma `HttpErrorResponse` do Quarkus em mensagem legível: cobre Bean Validation (`violations[]`), erros de negócio (`{ message }`), corpos vazios (mensagem por status 401/403/404/409/500) e falha de rede (status 0). É usado por praticamente toda tela ao tratar `subscribe({ error })`.

5. Autenticação no cliente — AuthService

O servidor guarda a sessão no cookie; o cliente apenas **espelha** o usuário atual em um signal. Todo o resto da aplicação lê esse signal de forma reativa.

```
@Injectable({ providedIn: 'root' })
export class AuthService {
  private readonly http = inject(HttpClient);
  private readonly baseUrl = `${API_BASE_URL}/auth`;

  readonly currentUser = signal<User | null>(null);
  readonly isAuthenticated = computed(() => this.currentUser() !== null);

  login(credentials) { return this.http.post<LoginResponse>(`${this.baseUrl}/login`, credentials); }
  logout() { return this.http.post<void>(`${this.baseUrl}/logout`, {})
    .pipe(tap(() => this.currentUser.set(null))); } // limpa o espelho
  fetchCurrentUser() { return this.http.get<User>(`${this.baseUrl}/me`)
    .pipe(tap((u) => this.currentUser.set(u))); } // atualiza o espelho

  getProfile() { return this.http.get<Profile>(`${this.baseUrl}/perfil`); } // Meu Perfil
  changePassword(request) { return this.http.post<LoginResponse>(`${this.baseUrl}/change-password`, request)
    .pipe(tap(() => { // zera mustChangePassword no signal
      const u = this.currentUser();
      if (u) this.currentUser.set({ ...u, mustChangePassword: false });
    })); }
  forgotPassword(email) { return this.http.post<void>(`${this.baseUrl}/forgot-password`, { email }); }
  resetPassword(request) { return this.http.post<LoginResponse>(`${this.baseUrl}/reset-password`, request); }
}
```

Padrão-chave: `currentUser` é a única fonte de verdade da sessão no cliente. Os guards a atualizam via `fetchCurrentUser()`; a diretiva `*appRole` e o layout a leem; o `logout()` a zera; e `changePassword()` a atualiza no lugar, apagando `mustChangePassword` — o que dispensa o modal bloqueante assim que a senha é definida.

LOGIN — O COMPONENTE

```
export class LoginComponent {
  readonly email = signal('');
  readonly password = signal('');
  readonly errorMessage = signal<string | null>(null);
  readonly loading = signal(false);

  onSubmit(): void {
    this.loading.set(true);
    this.authService.login({ email: this.email(), password: this.password() }).subscribe({
      next: (r) => r.authenticated
        ? this.router.navigate(['/dashboard']) // login sempre vai ao dashboard
        : this.errorMessage.set(r.message),
      error: () => this.errorMessage.set('Credenciais inválidas'),
    });
  }
}
```

Repare: o login **não** mais bifurca em "trocar senha". Mesmo com `mustChangePassword`, ele navega para `/dashboard`; é o layout autenticado que assume o controle e sobe o modal obrigatório (seção 15). A página de login ainda oferece o link "*Esqueci minha senha*" para `/esqueci-senha`.

6. Interceptor de credenciais

A API está em outra origem (8080) e a sessão é um cookie. Para o browser enviar esse cookie em requisições cross-origin, cada chamada precisa de `withCredentials: true`. Um interceptor funcional aplica isso a **todas** as requisições, de forma central:

```
export const credentialsInterceptor: HttpInterceptorFn = (req, next) => {
  return next(req.clone({ withCredentials: true }));
};
```

É o par natural da configuração de CORS do backend (`access-control-allow-credentials=true`): sem os dois lados alinhados, o cookie de sessão não trafegaria e toda requisição autenticada falharia com 401.

7. Guards de rota

Os guards são funções (`CanActivateFn`) e, em vez de confiar num flag local, **revalidam a sessão no backend a cada navegação** chamando `GET /auth/me`.

AUTHGUARD — EXIGE SESSÃO VÁLIDA

```
export const authGuard: CanActivateFn = () => {
  const authService = inject(AuthService);
  const router = inject(Router);
  return authService.fetchCurrentUser().pipe(
    map(() => true), // sessão ok → libera
    catchError(() => of(router.parseUrl('/login'))), // erro → redireciona
  );
};
```

Mudança importante: o `authGuard` **não redireciona mais** quando `mustChangePassword` é verdadeiro. O primeiro acesso deixou de ser uma rota (`/trocar-senha` foi **removida**, junto do antigo `changePasswordGuard`): agora é um **modal bloqueante** montado no layout. O guard só valida a existência de sessão.

ADMINGUARD — EXIGE PERFIL ADMIN

```
export const adminGuard: CanActivateFn = () => {
  ...
  return authService.fetchCurrentUser().pipe(
    map((user) => user.perfil === 'ADMIN' ? true : router.parseUrl('/acesso-negado')),
    catchError(() => of(router.parseUrl('/login'))),
  );
};
```

Três desfechos: autenticado e ADMIN → segue; autenticado sem permissão → `/acesso-negado` (antes voltava a `/dashboard`); não autenticado → login. Como efeito colateral útil, `fetchCurrentUser()` também popula o signal `currentUser`, deixando a UI pronta assim que a rota carrega.

8. Diretiva estrutural `*appRole`

Controla a **visibilidade visual** de elementos conforme o perfil, reagindo a login/logout sem código extra — porque observa o signal `currentUser` dentro de um `effect()`.

```
@Directive({ selector: '[appRole]', standalone: true })
export class RoleDirective {
  readonly appRole = input.required<string[] | string>();
  private rendered = false;

  constructor() {
    effect(() => {
      const roles = this.appRole();
      const allowed = Array.isArray(roles) ? roles : [roles];
      const user = this.auth.currentUser();
      this.toggle(!user && allowed.includes(user.perfil)); // mostra/esconde
    });
  }
  // toggle: cria ou limpa a embedded view via ViewContainerRef
}
```

Uso no template: `*appRole="['ADMIN']"`. Importante: isso é apenas *cosmético* — esconde botões, itens de menu, a busca/notificações do header e o formulário de ação da crise. A proteção real vem dos guards (front) e das anotações `@RolesAllowed` (back).

9. Roteamento e lazy loading

Todas as telas usam `loadComponent` (carregamento sob demanda). Há três rotas **públicas** (login e recuperação de senha) e o restante são *filhas* do `MainLayoutComponent`, que aplica `authGuard` uma vez para todas — e cada área administrativa reforça com `adminGuard`.

```
export const routes: Routes = [
  { path: '', redirectTo: 'login', pathMatch: 'full' },
  { path: 'login', loadComponent: () => import('.../login.component') ... },
  { path: 'esqueci-senha', loadComponent: () => import('.../forgot-password.component') ... }, // público
  { path: 'redefinir-senha', loadComponent: () => import('.../reset-password.component') ... }, // público (?to)
  {
    path: '',
    loadComponent: () => import('.../main-layout.component') ... ,
    canActivate: [AuthGuard], // protege todos os filhos
    children: [
      { path: 'dashboard', loadComponent: ... },
      { path: 'perfil', loadComponent: ... }, // Meu Perfil
      { path: 'acesso-negado', loadComponent: ... }, // destino do adminGuard
      { path: 'crises', loadComponent: ... },
      { path: 'crises/nova', loadComponent: ... }, // antes de :id
      { path: 'crises/:id', loadComponent: ... },
      { path: 'relatorios', canActivate: [adminGuard], loadComponent: ... }, // agora só ADMIN
      { path: 'relatorios/novo', canActivate: [adminGuard], loadComponent: ... },
      { path: 'usuarios' (+novo, +:id/editar), canActivate: [adminGuard], ... },
      // cadastros de referência (list + form novo/editar), todos com adminGuard:
      { path: 'departamentos' · 'tipos-crise' · 'instituicoes' · 'campi' · 'cenarios', canActivate: [adminGuard],
      { path: 'auditoria', canActivate: [adminGuard], loadComponent: ... }, // só ADMIN
    ],
  },
  { path: '**', redirectTo: 'login' },
];
```

Rota	Componente	Guard
/login	LoginComponent	público
/esqueci-senha	ForgotPasswordComponent	público
/redefinir-senha	ResetPasswordComponent	público
/dashboard	DashboardComponent	authGuard
/perfil	PerfilComponent	authGuard
/acesso-negado	AcessoNegadoComponent	authGuard
/crises · /crises/nova · /crises/:id	List · Form · Detail	authGuard
/relatorios · /relatorios/novo	List · Form	+ adminGuard
/usuarios · /usuarios/novo · /usuarios/:id/editar	List · Form	+ adminGuard
/departamentos · /tipos-crise · /instituicoes · /campi · /cenarios (+ novo/editar)	List · Form	+ adminGuard
/auditoria	AuditoriaListComponent	+ adminGuard

Ordem importa: crises/nova é declarada antes de crises/:id , senão "nova" seria capturada como um id. **Removida:** a rota /trocar-senha (e seu changePasswordGuard) não existe mais — substituída pelo modal de primeiro acesso.

10. Layout principal (MainLayoutComponent)

É a casca de todas as telas autenticadas. Foi redesenhado: uma **sidebar agrupada e recolhível**, um **header** com breadcrumbs, busca global, central de notificações e menu de perfil, além de hospedar o **modal bloqueante de primeiro acesso**. O `<router-outlet>` renderiza a página atual no centro.

SIDEBAR AGRUPADA E RECOLHÍVEL

A navegação é organizada em grupos com separadores e rótulos: **Dashboard**, **Gestão** (Crises, Nova Crise, Usuários), **Cadastros** (Instituições, Departamentos, Campi, Tipos de Crise, Cenários), **Relatórios**, **Sistema** (Auditoria) e **Conta** (Meu Perfil, Sair). Ela alterna entre **240 px** (expandida) e **72 px** (recolhida); o estado é persistido em `localStorage` (`sgci.sidebar.collapsed`). Quando recolhida, cada item mostra um tooltip via atributo `[data-tip]` (renderizado por `::after`). O item ativo é destacado por `routerLinkActive` .

Grupo	Itens	Visível para
Dashboard	Dashboard	Todos autenticados
Gestão	Crises, Nova Crise	Todos autenticados
Gestão	Usuários	ADMIN
Cadastros	Instituições, Departamentos, Campi, Tipos de Crise, Cenários	ADMIN
Relatórios	Relatórios	ADMIN
Sistema	Auditoria	ADMIN
Conta	Meu Perfil, Sair	Todos autenticados

Os blocos administrativos são envoltos por `*appRole="[ 'ADMIN' ]"` ; o COMUM vê apenas Dashboard, Crises, Nova Crise e Conta.

HEADER: BREADCRUMBS, BUSCA GLOBAL, NOTIFICAÇÕES E PERFIL

- **Breadcrumbs** reativos: reconstruídos a cada `NavigationEnd` traduzindo os segmentos da URL por um mapa `CRUMB_LABELS` (segmentos numéricos viram "Detalhe").
- **Busca global** (**ADMIN**), escondida por `*appRole` : `onSearch()` navega para `/crises` com `?q=<termo>` . O `CriseListComponent` lê esse `?q=` no `ngOnInit` e pré-preenche o filtro de busca.
- **Central de notificações** (**ADMIN**): um `computed` deriva os alertas de `DashboardResumo` — crises críticas, em aberto e em andamento. O resumo é carregado **uma vez por sessão do shell** (`dashboardService.resumo()` no construtor, com `catchError(of(null))`).
- **Menu de perfil** (todos): avatar com a inicial do nome → dropdown com *Meu Perfil* (`/perfil`) e *Sair*.

```
// os dois menus flutuantes fecham ao clicar em qualquer lugar do documento...
@HostListener('document:click') onDocumentClick() { this.closeMenus(); }
// ...e também a cada NavigationEnd (junto de fechar o menu mobile)
this.router.events.pipe(filter(isNavigationEnd), takeUntilDestroyed())
  .subscribe() => { this.updateCrumbs( ... ); this.mobileOpen.set(false); this.closeMenus(); });

onSearch() {
  // busca global (admin)
  const term = this.searchTerm().trim();
  this.router.navigate(['/crises'], term ? { queryParams: { q: term } } : {});
}

onLogout() { this.authService.logout().subscribe() => this.router.navigate(['/login']); }
```

MODAL BLOQUEANTE DE PRIMEIRO ACESSO

No fim do template do layout, condicionado ao signal do usuário:

```
@if (user()?.mustChangePassword) {
  <app-change-password-modal [mandatory]="true" (changed)="onFirstAccessChanged()" />
}
```

Enquanto `mustChangePassword` for verdadeiro, o modal cobre o app e não pode ser fechado. Ao concluir a troca, `changePassword()` zera o flag no signal (o modal some) e `onFirstAccessChanged()` navega para `/dashboard`. Detalhes na seção 15.

11. Features — visão geral e padrão comum

Cada feature segue a mesma anatomia: um `service` (`providedIn: 'root'` , usa `inject(HttpClient)` e deriva a URL de `API_BASE_URL`) e componentes de página com signals `loading` / `error` / dados carregados no `ngOnInit` . Erros passam por `extractApiError()` .

SERVIÇO TÍPICO (CRISES)

```
@Injectable({ providedIn: 'root' })
export class CriseService {
  private readonly baseUrl = `${API_BASE_URL}/api/crises`;
  listAll() { return this.http.get<Crise[]>(this.baseUrl, ...); }
  findById(id) { return this.http.get<Crise>(`.../${id}`, ...); }
  create(req) { return this.http.post<Crise>(this.baseUrl, req, ...); }
  changeStatus(id,s) { return this.http.patch(`.../${id}/status`, { status: s }, ...); }
  listAcoes(id) { return this.http.get(`.../${id}/acoes`, ...); }
  createAcao(id,req) { return this.http.post(`.../${id}/acoes`, req, ...); }
}
```

BLOCOS REUTILIZÁVEIS EM CORE/COMPONENTS

Além dos serviços por feature, o núcleo passou a oferecer primitivos compartilhados por várias telas (detalhados na seção 14 e 18):

- **ToastService** (signals) + **ToastContainerComponent** — notificações efêmeras globais, montadas uma vez no `App` .
- **ModalComponent** — overlay + cartão com projeção de conteúdo e input `closable` .
- **Gráficos SVG/CSS** — `PieChartComponent` , `LineChartComponent` , `BarChartComponent` (sem biblioteca externa).

PANORAMA DAS FEATURES

Feature	Páginas / componentes	Endpoints consumidos
auth	login · forgot-password · reset-password · change-password-modal	/auth/login · logout · me · perfil · change-password · forgot-password · reset-password
dashboard	dashboard (switch) · admin-dashboard	/api/dashboard/resumo + fontes ADMIN (ver seção 17)
perfil	perfil	/auth/perfil
acesso-negado	acesso-negado	—
crises	list · form · detail	/api/crises (+ status, acoes)
relatorios	list · form ADMIN	/api/relatorios
usuarios	list · form ADMIN	/api/usuarios
auditoria	list ADMIN	/api/auditoria (+ facets)
departamentos · tipos-crise · instituicoes · campi · cenarios	list · form ADMIN	/api/{ref}

Estado atual: os cadastros de referência e o novo módulo **usuarios** ganharam telas de formulário (rotas `/novo` e `/:id/editar`) e agora estão inteiramente sob `adminGuard`. As telas de **crises** continuam acessíveis a todos os autenticados (com regras por perfil dentro delas — ver seção 18).

12. Feature Crises em profundidade

12.1 Formulário de nova crise

É a tela mais rica: carrega em paralelo as quatro fontes dos selects e monta o `CriseRequest`. O `nivel` começa em `BAIXO` e as FKs começam nulas.

```
export class CriseFormComponent implements OnInit {
  readonly tipos = signal<TipoCrise[]>([]);  readonly campi = signal<Campus[]>([]);
  readonly cenarios = signal<Cenario[]>([]);  readonly departamentos = signal<Departamento[]>([]);
  readonly niveis: NivelCriticidade[] = ['BAIXO', 'MEDIO', 'ALTO', 'CRITICO'];

  model: CriseRequest = { titulo:'', descricao:'', nivel:'BAIXO',
    tipoCriseId:null, campusId:null, cenarioId:null, departamentoId:null };

  ngOnInit() {                                     // popula os 4 dropdowns
    this.tipoCriseService.listAll().subscribe((d) => this.tipos.set(d));
    this.campusService.listAll().subscribe((d) => this.campi.set(d));
    this.cenarioService.listAll().subscribe((d) => this.cenarios.set(d));
    this.departamentoService.listAll().subscribe((d) => this.departamentos.set(d));
  }
  onSubmit() {
    this.criseService.create(this.model).subscribe({
      next: () => this.router.navigate(['/crises']),
      error: () => this.error.set('Não foi possível cadastrar a crise ... '),
    });
  }
}
```

Criar crise é permitido a **todos os autenticados** (inclusive COMUM). No backend, a crise passa a ter **dono** (`created_by` , extraído do JWT); o COMUM só enxerga/acessa as próprias, enquanto o ADMIN vê todas (ver seções 17 e 18).

12.2 Detalhe da crise — máquina de estados no cliente

A tela de detalhe implementa o ciclo de vida da crise usando um mapa `fluxo` e dois `computed` : um para o próximo status, outro para decidir se o formulário de nova ação aparece.

```
private readonly fluxo: Record<StatusCrise, StatusCrise | null> = {
  ABERTA: 'EM_ANDAMENTO', EM_ANDAMENTO: 'RESOLVIDA',
  RESOLVIDA: 'ENCERRADA', ENCERRADA: null,           // terminal
};
readonly proximoStatus = computed(() => {
  const atual = this.crise();
  return atual ? this.fluxo[atual.status] : null;
});
readonly formVisivel = computed(() => {           // só registra ação se aberta/andamento
  const s = this.crise()?.status;
  return s === 'ABERTA' || s === 'EM_ANDAMENTO';
});
```

No template, tanto o bloco de avançar status quanto o formulário de nova ação ficam sob `*appRole="['ADMIN']"` — ou seja, o COMUM vê o detalhe e a timeline, mas **somente leitura**. Registrar ação (`POST /api/crises/{id}/acoes`) e mudar status são exclusivos do ADMIN.

Essa máquina de estados é o *espelho* da regra do backend (crise ENCERRADA é terminal); o front esconde os controles e o back garante de fato — defesa em profundidade. O sucesso das ações usa o `ToastService`.

13. Dashboard por perfil, Relatórios e Auditoria

DASHBOARD — UM SWITCH POR PERFIL

O `DashboardComponent` deixou de ser uma página única e virou um **switch por perfil**:

```
readonly isAdmin = computed(() => this.user()?.perfil === 'ADMIN');

// template
@if (isAdmin()) {
  <app-admin-dashboard />                                // Centro de Monitoramento (seção 17)
} @else {
  // home enxuta do COMUM: boas-vindas + KPIs das próprias crises + últimas crises
}
```

Para o **COMUM**, o componente carrega `criseService.listAll()` (que o backend já filtra pelas crises do próprio usuário) e computa uma home enxuta: cartão de boas-vindas, KPIs simples (`totalCrises`, `emAberto` = ABERTA+EM_ANDAMENTO, `encerradas` = RESOLVIDA+ENCERRADA) e as `ultimasCrises` (5 primeiras). Para o **ADMIN**, o componente não busca nada — delega tudo ao `AdminDashboardComponent`, que carrega seus próprios dados (seção 17).

RELATÓRIOS **SÓ ADMIN**

Toda a área de relatórios agora exige `adminGuard` (o COMUM não a acessa nem vê o link). No formulário, o select de crises continua **filtrado** para exibir apenas crises `RESOLVIDA` / `ENCERRADA` (`crisesElegiveis`), espelhando a regra do backend de que só crises fechadas geram relatório.

AUDITORIA **SÓ ADMIN**

Lista com filtros por entidade, ação e usuário — os valores dos dropdowns vêm de `GET /api/auditoria/facets`. Usa `DatePipe` para a data/hora e helpers que coloquem badges por ação (CRIAR→verde, ATUALIZAR→azul, EXCLUIR→vermelho) e por faixa de status HTTP. Além da própria tela, os registros de auditoria alimentam o painel "Últimas atividades" do Centro de Monitoramento.

14. Primitivos de UX (Toast, Modal, Skeleton)

Para padronizar feedback e evitar dependências externas, o núcleo ganhou um pequeno conjunto de primitivos reutilizáveis em `core/components` e `core/services`, apoiados por estilos globais em `styles.scss` (`.toast-*`, `.modal-*`, `.skeleton`, `.field-error`).

TOASTS — TOASTSERVICE (SIGNALS) + TOASTCONTAINERCOMPONENT

O serviço mantém a fila em um signal e oferece `success()`, `error()` e `info()`, cada um com auto-dismiss por `setTimeout` (4 s / 6 s). O container é montado uma única vez no `App` e renderiza a pilha no canto superior direito.

```
@Injectable({ providedIn: 'root' })
export class ToastService {
  private sequence = 0;
  readonly toasts = signal<Toast[]>([]); // { id, type, message }
  success(m, t = 4000) { this.push('success', m, t); }
  error(m, t = 6000) { this.push('error', m, t); }
  info(m, t = 4000) { this.push('info', m, t); }
  dismiss(id) { this.toasts.update(l => l.filter(x => x.id !== id)); }
  private push(type, message, ms) {
    const id = ++this.sequence;
    this.toasts.update(l => [...l, { id, type, message }]);
    if (ms > 0) setTimeout(() => this.dismiss(id), ms);
  }
}
```

MODAL GENÉRICO — MODALCOMPONENT

Overlay + cartão centralizado com projeção de conteúdo (`<ng-content>`). Inputs: `title` e `closable` (padrão `true`). Emite (`close`) ao clicar no botão de fechar ou no backdrop — mas **quando `closable=false` ignora o backdrop e esconde o botão**, viabilizando o fluxo obrigatório de primeiro acesso.

```
<div class="modal-overlay" (click)="onBackdrop()">
  <div class="modal-card" role="dialog" aria-modal="true" (click)="$event.stopPropagation()">
    <header class="modal-card__head">
      <h2>{{ title() }}</h2>
      @if (closable()) { <button (click)="close.emit()">X</button> }
    </header>
    <div class="modal-card__body"><ng-content /></div>
  </div>
</div>
// onBackdrop(): só emite (close) se closable()
```

SKELETON E ESTILOS DE CAMPO

Não é um componente, e sim uma classe utilitária global (`.skeleton`) usada para placeholders animados durante o carregamento, junto de `.field-error` / `.field-hint` para mensagens de formulário.

15. Primeiro acesso (modal bloqueante) e recuperação de senha

PRIMEIRO ACESSO — MODAL OBRIGATÓRIO

Quando o backend marca `mustChangePassword`, o usuário faz login normalmente e cai no `/dashboard`; o layout então monta `<app-change-password-modal [mandatory]="true">`. Em modo obrigatório, o modal **não é fechável** (sem botão de cancelar, backdrop inerte) e exibe uma mensagem explicativa de primeiro acesso.

```

export class ChangePasswordModalComponent {
  readonly mandatory = input(false);
  readonly changed = output<void>();  readonly closed = output<void>();
  // título muda conforme o modo
  readonly title = computed(() => this.mandatory() ? 'Definição de senha obrigatória' : 'Alterar senha');

  onSubmit() {
    if (!this.currentPassword()) { this.errorMessage.set('Informe a senha atual.');
```

Como `changePassword()` zera `mustChangePassword` no signal, o `@if` do layout deixa de renderizar o modal automaticamente; em seguida `(changed)` dispara `onFirstAccessChanged()`, que navega para `/dashboard`. O mesmo componente, com `mandatory=false`, é reaproveitado em "Meu Perfil" (seção 16) com botão de cancelar.

RECUPERAÇÃO DE SENHA — PÁGINAS PÚBLICAS

Duas telas públicas (fora do layout autenticado) cobrem o fluxo de esquecimento:

- **/esqueci-senha** (`ForgotPasswordComponent`): coleta o e-mail e chama `forgotPassword(email)`. A resposta é sempre **neutra** — não revela se a conta existe: sucesso e falhas de servidor mostram a mesma tela de confirmação; apenas erro `400` (e-mail malformatado) volta ao formulário com mensagem.
- **/redefinir-senha** (`ResetPasswordComponent`): lê `?token=` da query string no `ngOnInit` (sinaliza `tokenMissing` se ausente), valida a nova senha pela mesma política e chama `resetPassword({ token, newPassword })`. No sucesso, exibe confirmação e um toast, com link para voltar ao login; se o link expirou, mostra o erro via `extractApiError`.

No backend, o token é um hash SHA-256 guardado com validade de 30 min e o link chega por e-mail — do lado do front, tudo o que importa é ler o `token` da URL e respeitar a política de senha.

16. Meu Perfil

A rota `/perfil` (`PerfilComponent`) exibe os dados cadastrais do usuário em modo **somente leitura**. No `ngOnInit` chama `authService.getProfile()` (GET `/auth/perfil`) e mostra: **Nome**, **E-mail**, **Cargo**, **Departamento** (`departamentoNome`), **Instituição** (`instituicaoNome`) e **Perfil** (rotulado como "Administrador" ou "Usuário").

```
ngOnInit() {
  this.authService.getProfile().subscribe({
    next: (profile) => { this.profile.set(profile); this.loading.set(false); },
    error: (err) => { this.error.set(extractApiError(err, 'Não foi possível carregar o perfil.')); ... },
  });
}
// botão "Alterar senha" abre o mesmo modal, agora não obrigatório
openPasswordModal() { this.showPasswordModal.set(true); }
closePasswordModal() { this.showPasswordModal.set(false); }
```

O botão **"Alterar senha"** monta o `ChangePasswordModalComponent` com `mandatory=false` — assim ele é fechável (botão Cancelar + backdrop) e reaproveita toda a validação de política e o feedback via toast.

17. Centro de Monitoramento (admin) — `AdminDashboardComponent`

Para o ADMIN, o dashboard é um **centro de monitoramento** que agrega o estado do sistema em uma única tela densa. Toda a informação é **computada no cliente** a partir de **endpoints ADMIN já existentes** — nenhuma API nova foi criada.

CARGA RESILIENTE EM PARALELO (`FORKJOIN`)

No `ngOnInit`, nove fontes são buscadas em paralelo; cada uma tem seu próprio `catchError` (para `of([])` ou `of(null)`), de modo que a falha de uma fonte não derruba o painel inteiro.

```
forkJoin({
  resumo: this.dashboardService.resumo().pipe(catchError(() => of(null))),
  crises: this.criseService.listAll().pipe(catchError(() => of([]))),
  usuarios: this.usuarioService.listAll().pipe(catchError(() => of([]))),
  tipos: this.tipoCriseService.listAll().pipe(catchError(() => of([]))),
  departamentos: this.departamentoService.listAll().pipe(catchError(() => of([]))),
  campi: this.campusService.listAll().pipe(catchError(() => of([]))),
  instituicoes: this.instituicaoService.listAll().pipe(catchError(() => of([]))),
  relatorios: this.relatorioService.listAll().pipe(catchError(() => of([]))),
  auditoria: this.auditoriaService.listAll().pipe(catchError(() => of([]))),
}).subscribe({ next: (r) => { ... seta os signals ... ; this.atividades.set(r.auditoria.slice(0, 7)); }, ... });
```

KPIS E MÉTRICAS DERIVADAS

Cada indicador é um `computed` sobre os signals carregados:

Métrica	Como é derivada no cliente
Total	<code>crises().length</code>
Em andamento / Abertas	filtros por <code>status</code> (abertas = ABERTA + EM_ANDAMENTO)
Críticas ativas	<code>nível === 'CRITICO'</code> e ainda não resolvida/encerrada
Resolvidas	status RESOLVIDA + ENCERRADA
Taxa de resolução	<code>round(resolvidas / total × 100)</code>
Tempo médio	média de (<code>relatorio.createdAt - crise.createdAt</code>) das crises com relatório; "—" se nenhuma. Rótulo em horas (<1 dia) ou dias.
Tendência do total	novas crises deste mês vs. mês anterior (up/down/flat)

PAINEL DE ALERTAS

- **Relatórios pendentes:** crises resolvidas/encerradas *sem* relatório (`relatorioPorCrise` não contém a crise).
- **Sem responsável:** crises com `createdBy === null` (legado pré-dono).
- **Usuários inativos** (`!u.active`) e **pendências** (crises ABERTA).

GRÁFICOS, ATIVIDADES, AÇÕES RÁPIDAS E TABELA

- **Gráficos SVG/CSS próprios** (seção 18): evolução mensal (linha, 6 meses), por categoria/tipo (pizza), por status (rosca) e por departamento (barras horizontais). A paleta é fixa (`PALETTE`) e cores de status padronizadas.
- **Últimas atividades:** os 7 registros mais recentes de `/api/auditoria`, com ícone por ação (CRIAR→plus, ATUALIZAR→edit, EXCLUIR→trash).
- **Ações rápidas:** atalhos (ex.: nova crise, cadastros) e **tabela de últimas crises** com busca, filtro por status/nível, ordenação (recentes/antigas/nível/título) e paginação (`pageSize = 6`), tudo em `computed` encadeados (`filtered` → `totalPages` → `safePage` → `paged`).

Layout denso: a página usa um grid mestre de **12 colunas** (`.dash-grid`, `.col-4/6/8/12`) com `max-width: 1600px`; os KPIs são cartões compactos horizontais. A antiga **"faixa de situação" (Estado Geral)** foi removida a pedido do usuário — suas métricas seguem vivas nos KPIs e no painel de alertas.

18. Gráficos em SVG/CSS sem biblioteca

Todos os gráficos são **componentes próprios** em `core/components/charts`, desenhados com SVG e CSS — **sem Chart.js, D3 ou similar**. Compartilham os tipos de `chart.types.ts` (`ChartSlice`, `ChartPoint`, `ChartBar`) e recebem dados via `input()`, recomputando a geometria em `computed`. Os estilos são globais e namespaced (`.chart-*` em `styles.scss`), já que esses componentes não têm SCSS próprio.

Componente	Tipo	Como funciona
<code>PieChartComponent</code>	Pizza / rosca	Desenha fatias como <code><path></code> (arco polar). Input <code>size</code> dimensiona o SVG; <code>innerRadius</code> (0–1) > 0 vira rosca com furo central e valor/rótulo centrais. Inclui legenda com valor e %.
<code>LineChartComponent</code>	Linha + área	<code>viewBox 900x200</code> (responsivo), grade horizontal em 3 níveis, área com gradiente (id único por instância) e pontos rotulados no eixo X.
<code>BarChartComponent</code>	Barras horizontais	CSS puro (<code></code> + larguras em %); normaliza cada barra pelo maior valor. Ideal para ranking por categoria/departamento.

```
// pie-chart: pizza (innerRadius=0) ou rosca (0<innerRadius<1)
<app-pie-chart [slices]="porStatus()" [innerRadius]="0.6" [size]="132"
               centerValue="..." centerLabel="..." />
// line-chart e bar-chart
<app-line-chart [points]="evolucaoMensal()" color="#2f9e41" />
<app-bar-chart [bars]="porDepartamento()" />
```

Estado vazio é tratado dentro de cada componente ("Sem dados para exibir." / legenda "Sem dados no período."), e todos expõem `aria-label` descritivo para acessibilidade.

19. Acesso Negado e endurecimento do perfil COMUM

PÁGINA /ACESSO-NEGADO

Componente enxuto (template e estilos inline) exibido quando o `adminGuard` barra um usuário autenticado sem permissão. Mostra um ícone, a mensagem "Esta seção é restrita a administradores." e um link "Voltar ao início" para `/dashboard`.

ENDURECIMENTO DO COMUM

O perfil COMUM foi restringido em três frentes, sempre com defesa em profundidade (front cosmético + backend autoritativo):

- **Detalhe da crise = somente leitura:** o formulário de nova ação e o botão de avançar status ficam sob `*appRole="['ADMIN']"`. O backend tornou `POST /api/crises/{id}/acoes @RolesAllowed("ADMIN")`; o COMUM ainda lê a timeline (`GET .../acoes segue @Authenticated`).
- **Sem Relatórios:** o link some da sidebar (`*appRole`) e a rota `/relatorios` exige `adminGuard`; no backend o controller inteiro é `@RolesAllowed("ADMIN")`.
- **Só as próprias crises:** crises têm dono (`created_by`). O `CriseService` do backend filtra a listagem por dono para o COMUM e bloqueia o acesso ao detalhe de crise alheia com `ForbiddenException`; o ADMIN vê todas.

20. Fluxos de ponta a ponta

Autenticação e proteção de rota

- 1 Usuário envia login → AuthService.login() → POST /auth/login
- 2 Backend responde com Set-Cookie: jwt (HttpOnly) → navega SEMPRE para /dashboard
- 3 A cada navegação guardada:
authGuard → fetchCurrentUser() → GET /auth/me (cookie enviado pelo interceptor)
├─ sucesso → popula currentUser + libera
└─ erro → redireciona a /login
- 4 adminGuard (áreas ADMIN): exige perfil === 'ADMIN', senão vai a /acesso-negado
- 5 UI reativa: *appRole e o layout leem o signal currentUser
- 6 Logout → POST /auth/logout → currentUser.set(null) → /login

Primeiro acesso (modal bloqueante)

- 1 Login de usuário com mustChangePassword=true → vai para /dashboard normalmente
- 2 Layout detecta user().mustChangePassword → monta <app-change-password-modal [mandatory]="true"> (não fechável: sem Cancelar, backdrop inerte)
- 3 Usuário define nova senha (política: 8+ com letra e número) → changePassword()
→ POST /auth/change-password
- 4 Sucesso → toast + currentUser recebe mustChangePassword=false → @if some com o modal
- 5 (changed) dispara onFirstAccessChanged() → navega para /dashboard já liberado

Recuperação de senha (esqueci → e-mail → redefinir → login)

- 1 /esqueci-senha: informa e-mail → forgotPassword(email) → POST /auth/forgot-password (resposta NEUTRA – não revela se a conta existe; só 400 volta ao form)
- 2 Backend gera token (SHA-256, validade 30 min) e envia link por e-mail
- 3 Usuário abre o link → /redefinir-senha?token=... (ngOnInit lê o ?token=)
- 4 Define nova senha (mesma política) → resetPassword({ token, newPassword })
→ POST /auth/reset-password
- 5 Sucesso → toast + tela de confirmação com link → /login

COMUM só enxerga as próprias crises

- 1 COMUM autentica → /dashboard mostra a home enxuta (KPIs das próprias crises)
- 2 /crises → criseService.listAll() → backend filtra por created_by do JWT
- 3 Abre /crises/:id própria → detalhe + timeline em SOMENTE LEITURA (form de ação e avançar status escondidos por *appRole=["ADMIN'])
- 4 Tenta crise alheia ou /relatorios → backend 403 / adminGuard → /acesso-negado
- 5 Pode criar crise (POST /api/crises grava created_by) – passa a ser dona dela

Operação sobre uma crise (ADMIN)

- 1 ADMIN em /crises/nova preenche o form (4 selects carregados em paralelo)
→ CriseService.create() → POST /api/crises → volta a /crises
- 2 Abre /crises/:id (detalhe + ações)

- 3 Registrar ação (form visível só se ABERTA/EM_ANDAMENTO e só ADMIN)
→ createAcao() → POST /api/crises/{id}/acoes → toast de sucesso
- 4 ADMIN avança status (computed proximoStatus)
→ changeStatus() → PATCH /api/crises/{id}/status
- 5 Após RESOLVIDA/ENCERRADA: /relatorios/novo lista só crises elegíveis